

Lecture 7 Classification Evaluation, Choice of Divergence, Regularization

Recap: Confusion Matrix for Binary Classification

Given a dataset of size 100, we might arrive at the following confusion matrix

	Predicted Positive ($\hat{y} = 1$)	Predicted negative ($\hat{y} = 0$)
Positive Class $y = 1$	31	20
Negative Class $y = 0$	14	35

	Predicted Positive ($\hat{y} = 1$)	Predicted negative ($\hat{y} = 0$)
Positive Class $y = 1$	True positive (TP)	False Negative (FN)
Negative Class $y = 0$	False positive (FP)	True Negative (TN)

Sensitivity and Specificity

	Predicted Positive ($\hat{y} = 1$)	Predicted negative ($\hat{y} = 0$)
Positive Class $y = 1$	True positive (TP)	False Negative (FN)
Negative Class $y = 0$	False positive (FP)	True Negative (TN)

- **Sensitivity** = $\frac{TP}{\text{positive class}} = \frac{TP}{TP+FN}$
 - AKA **recall**, true positive rate
- **Specificity** = $\frac{TN}{\text{negative class}} = \frac{TN}{FP+TN}$
 - True negative rate
- Can combine these to get a single measure called balanced accuracy
 - Balanced accuracy = $\frac{1}{2}$ (specificity + sensitivity)

Precision and Recall

	Predicted Positive ($\hat{y} = 1$)	Predicted negative ($\hat{y} = 0$)
Positive Class $y = 1$	True positive (TP)	False Negative (FN)
Negative Class $y = 0$	False positive (FP)	True Negative (TN)

- **Precision** = $\frac{TP}{\text{predicted positive}} = \frac{TP}{TP+FP}$
- Fraction predicted positive truly positive
- **Recall** = $\frac{TP}{\text{positive class}} = \frac{TP}{TP+FN}$ (True positive rate)
- Precision and recall are useful when the negatives don't matter
- Harmonic mean of precision and recall

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}}$$

Receiver Operating Characteristic (ROC)

In binary classification, the ROC curve plots the **true positive rate** (TPR) and **false positive rate** (FPR) as we vary the threshold for labeling a positive sample.

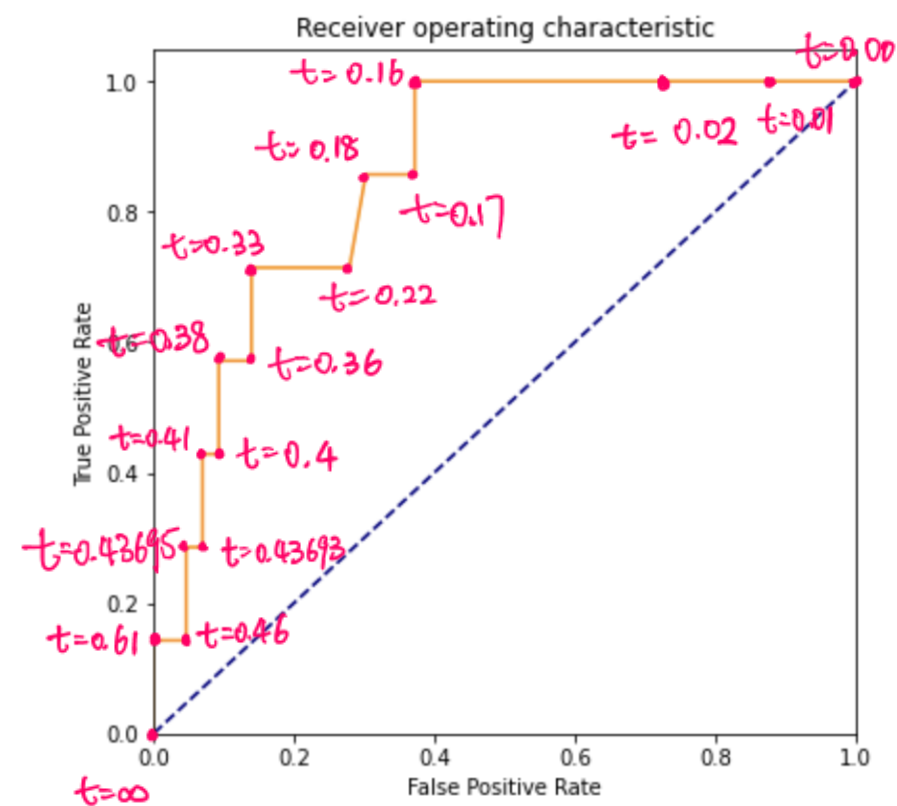
- **TPR** = sensitivity = $\frac{TP}{\text{positive class}} = \frac{TP}{TP+FN}$
- **FPR** = 1 - specificity = $1 - \frac{TN}{\text{negative class}} = \frac{FP}{FP+TN}$

	Predicted Positive ($\hat{y} = 1$)	Predicted negative ($\hat{y} = 0$)
Positive Class $y = 1$	True positive (TP)	False Negative (FN)
Negative Class $y = 0$	False positive (FP)	True Negative (TN)

ROC Visualization

Properties of ROC:

- Bottom left corner: predict only negatives: $TPR=FPR=0$
- Top right corner: predict only positives: $TPR=FPR=1$
- Blue diagonal:
 - Randomly guessing “positive” with $p=TPR$
- The ideal classifier
 - Top corner: $TPR=1, FPR=0$



	$\hat{y} = 1$	$\hat{y} = 0$
$y = 1$	TP	FN
$y = 0$	FP	TN

Are the following statements correct?



Classifiers with the same accuracy are guaranteed to have the same sensitivity and specificity

0

It is always a good idea to decompose accuracy

0

Varying the threshold of the predicted label in a classification algorithm can change the sensitivity and specificity of the model, and it does not change the parameters of the model

0

ROC curve captures trade-off between sensitivity and specificity as the threshold is varied, and it lies between ideal and random

0

Area Under the ROC Curve (AUC-ROC)

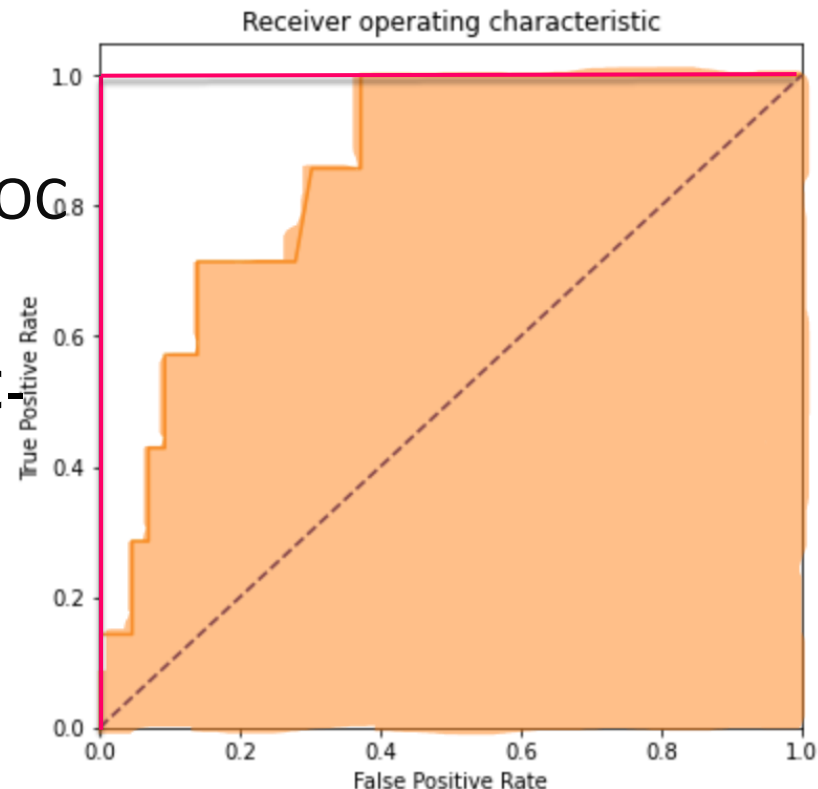
AUC: a single measure of classifier performance

- The **ideal classifier** has an AUC-ROC of 1
- The random classifier has an AUC-ROC of 0.5

AUC-ROC: 0.8555

AUC is a unit-free measure of classification

- Similar to R^2 for linear regression



are the following statements correct?

 0

For a random positive observation and a random negative observation, the AUC is the likelihood that the model correctly differentiates which is which

0

AUC measures the model's discriminative ability—that is, the ability to differentiate between samples with positive and negative labels

0

Multiclass Generalizations

- Define each class in turn to be “positive” ($y = 1$) and all other classes to be negative ($y = 0$)
- Compute sensitivity and specificity for each “positive” class using the usual formulas.
- Can combine binary metrics in many ways.

Multiclass Generalizations

- Can combine binary metrics in many ways:
- **macro**: average binary one-vs-all metrics for each class

$$\text{precision}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \frac{TP_k}{TP_k + FP_k}$$

- **micro**: average binary metrics across classes

$$\text{precision}_{\text{micro}} = \frac{\sum_{k=1}^K TP_k}{\sum_{k=1}^K (TP_k + FP_k)}$$

Recap

We learn a supervised learning algorithm by minimizing the loss function

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^N \text{div}(f_{\theta}(X_i), y_i)$$

- Average divergence between true and desired outputs over “training” inputs
- Approximation to “true” risk: expected divergence between desired and true outputs

Recap: Training a ML Algorithm

The diagram illustrates the components of the total loss function $\ell(\theta)$. It features several hand-drawn boxes with arrows pointing to the formula:

- total loss**: Points to the entire formula $\ell(\theta) = \frac{1}{N} \sum_{i=1}^N \text{div}(f_{\theta}(X_i), y_i)$.
- Average over all training instances**: Points to the fraction $\frac{1}{N}$.
- Divergence btw desired output and actual output for a given input X_i** : Points to the div function.
- output of a ML in response to input X_i** : Points to $f_{\theta}(X_i)$.
- desired output in response to input X_i** : Points to y_i .

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^N \text{div}(f_{\theta}(X_i), y_i)$$

Below the formula, the optimization goal is stated in a pink box:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \ell(\theta)$$

- Define a total “loss” over all training instances
- Quantifies the difference between desired output and the actual output, as a function of the weights
- Find the weights that minimize the loss

Recap

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^N \text{div}(f_{\theta}(X_i), y_i)$$

We minimize it through gradient descent

- Iterative updates against the gradient of the loss w.r.t. θ

$$\nabla_{\theta} \ell(\theta) = \frac{1}{N} \sum_{i=1}^N \underbrace{\nabla_{\theta} \text{div}(f_{\theta}(X_i), y_i)}_{\text{computed through autograd}}$$

solved through
gradient descent
as

$$\boxed{\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \ell(\theta)} \longrightarrow \boxed{\theta_k = \theta_{k-1} - \eta_k \nabla_{\theta} \ell(\theta)}$$

Training Gradient Descent is an Art

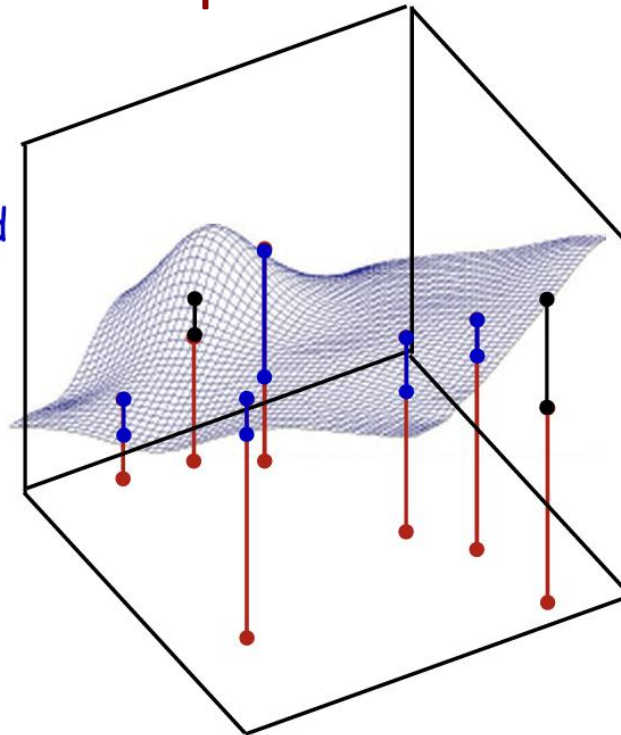
- Vanilla GD easily gets stuck at local minima with small enough step sizes
- Quality of the converged solution depends on the loss surface shape, the initial position, and the step sizes
- Online methods that present training instances incrementally make quicker updates
 - “Stochastic Gradient Descent” updates parameters after individual randomly chosen instances
 - “minibatch descent” updates them after minibatches of randomly chosen instances
 - **Require shrinking learning rates to converge**

Moving on: Topics for the day

- Divergence metric
- Overfitting

General Approach to Supervised Learning

Blue lines: error when function is *below* desired output



Black lines: error when function is *above* desired output

$$E = \sum_i (d_i - f(\mathbf{x}_i, \mathbf{W}))^2$$

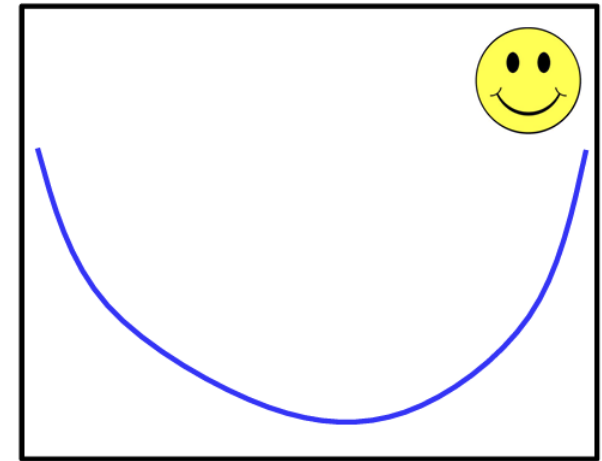
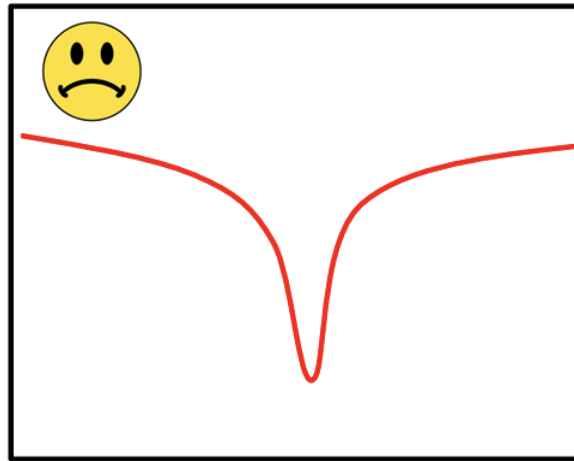
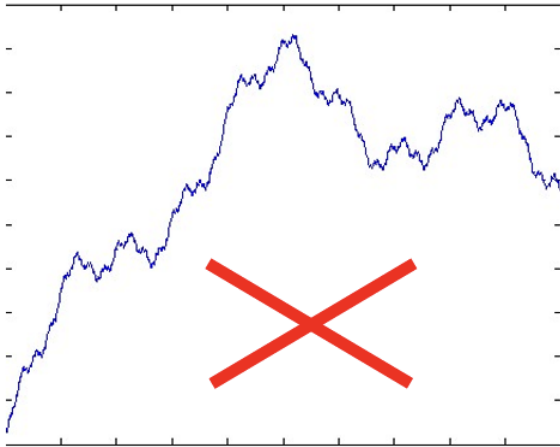
- Define a **divergence** between the actual output for any parameter value and the desired output
 - Typically L2 divergence or KL divergence
 - In linear regression, MLE = L2 divergence
 - In logistic regression, MLE = KL divergence

General Approach to Supervised Learning

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^N \text{div}(f_{\theta}(X_i), y_i)$$

- The convergence of the gradient descent depends on the divergence
- Ideally, must have a shape that results in a significant gradient in the right direction outside the optimum
 - To “guide” the algorithm to the right solution

Desiderata for a Good Divergence



Must be smooth and not have many poor local optima

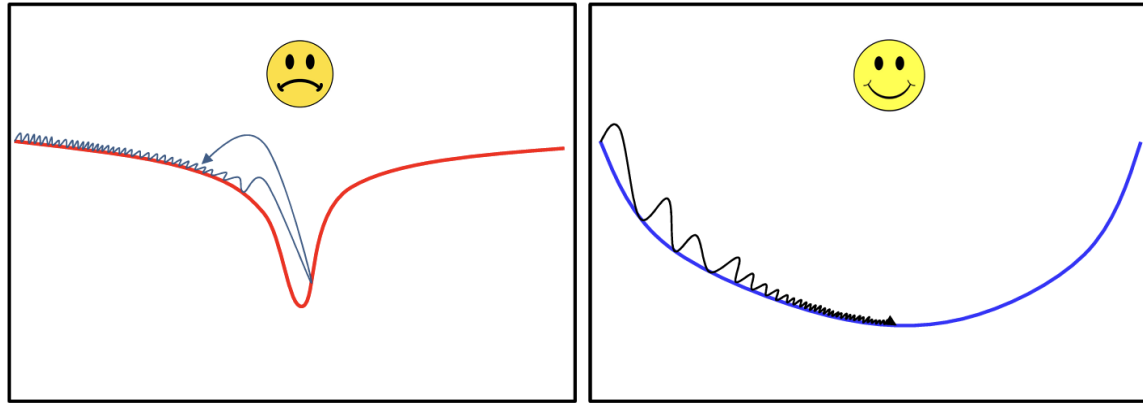
Low slopes far from the optimum = bad

- Initial estimates far from the optimum will take forever to converge

High slopes near the optimum = bad

- Steep gradients

Desiderata for a Good Divergence



Functions that are **shallow** when far from the optimum will result in very small steps during optimization

- Slow convergence of gradient descent

Functions that are **steep** near the optimum will result in large steps and overshoot during optimization

- Gradient descent will not converge easily

The best type of divergence is **steep when far from the optimum, but shallow at the optimum**

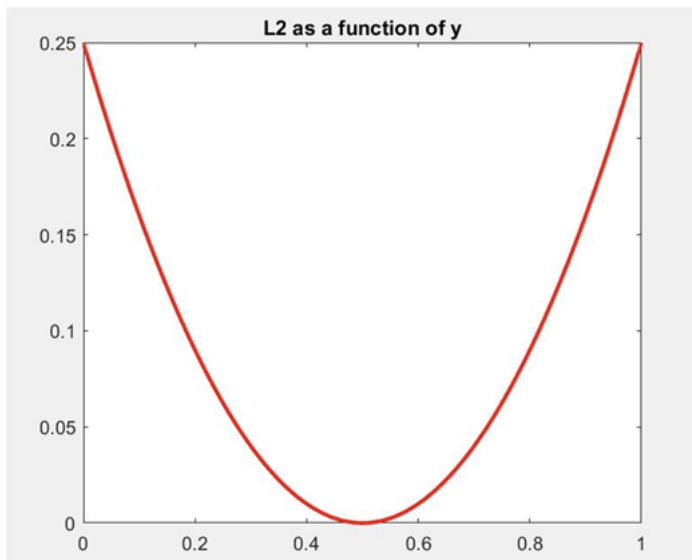
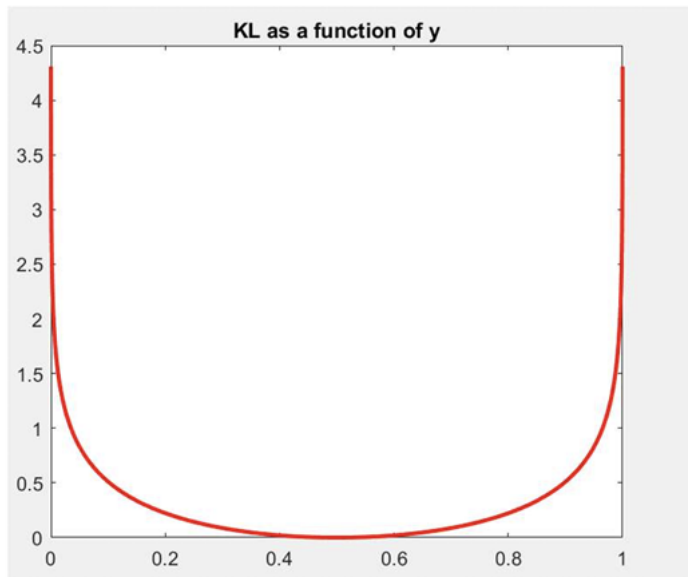
- But not too shallow: ideally quadratic in nature

Choices for Divergence in Supervised Learning

regression	classification
Desired output: d	Desired output: $[0, 0, \dots, 1, \dots, 0]$
L2 $Div = \frac{1}{2}(y - d)^2$	$Div = \frac{1}{2} \sum_i (y_i - d_i)^2$
KL $Div = -y \log(d) - (1 - y) \log(1 - d)$	$Div = \sum_i y_i \log(d_i) - \sum_i (1 - y_i) \log(1 - d_i)$

- Most common choices: The L2 divergence and the KL divergence
- L2 is popular for ML algorithms that perform numeric prediction/regression
- KL is popular for ML algorithms that perform classification

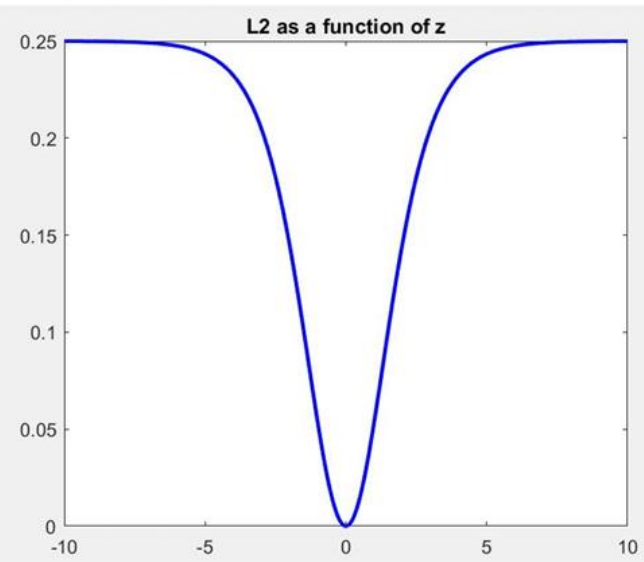
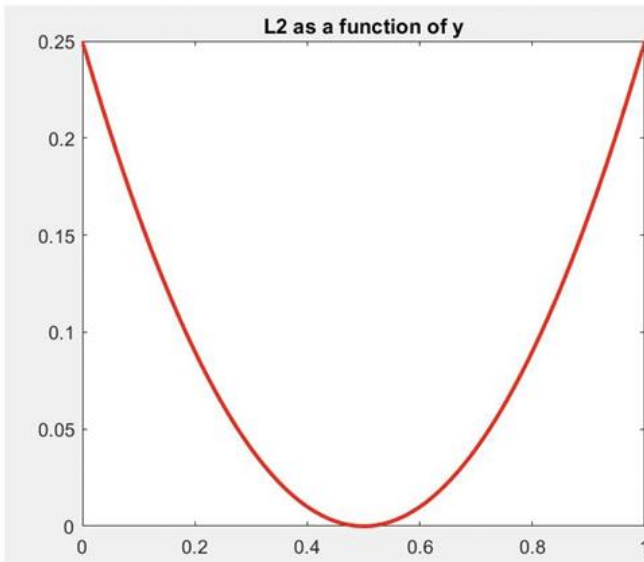
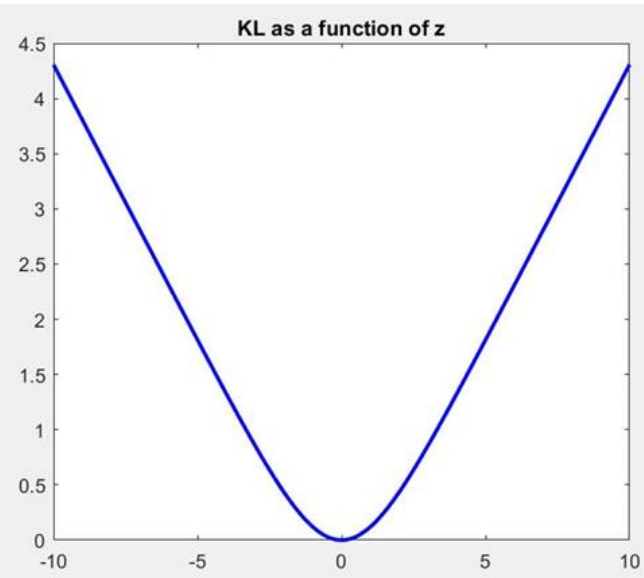
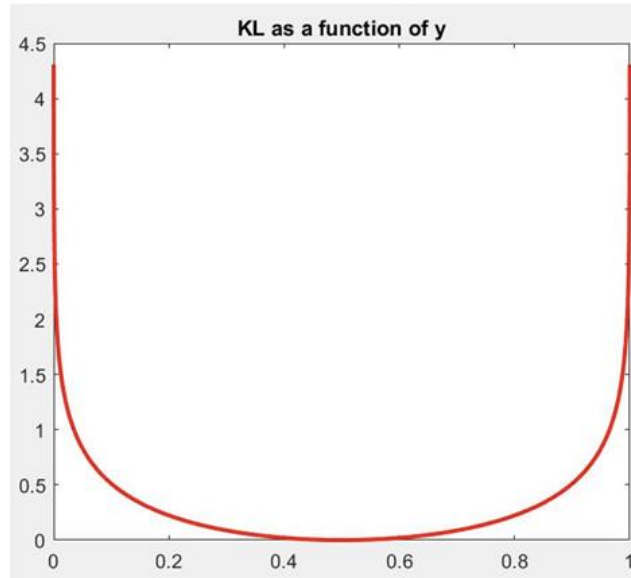
L2 or KL?



- We can also compute L2 divergences between target and actual output probabilities for classification
- Figure shows L2 and KL divergences for a target output of $p = 0.5$, as a function of sigmoid output y .
- Both are convex, and L2 may appear more bowl-like and “nice” (KL appears to flatten badly near the minimum)

L2 or KL?

But as a function of the argument z of the sigmoid, only one of them is convex



L2 or KL?

- The L2 divergence has long been favored in most applications
- It is particularly appropriate when attempting to perform regression
 - Numeric prediction
- The KL divergence is better when the intent is classification
 - The output is a probability vector

Gradient Descent Revisited

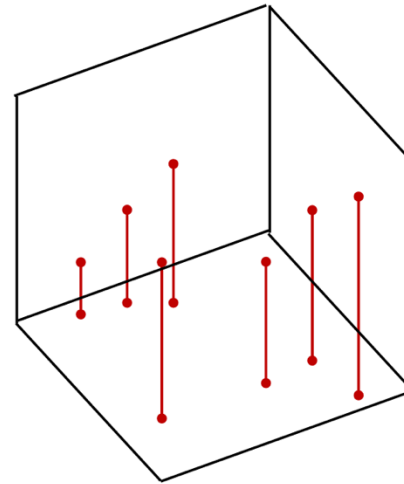
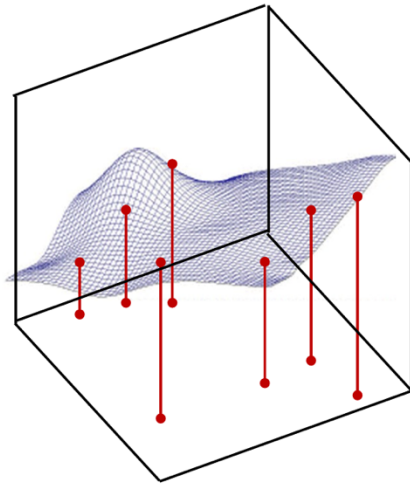
- Gradient descent can be sped up by incremental updates (stochastic GD)
- Convergence can be improved using smoothed updates (GD clipping: taking the average of the gradient in the last few steps)
- The choice of divergence affects both the learned parameters and the results

The Problem of Data Underspecification

- The figures shown to illustrate the learning problem so far were fake news...

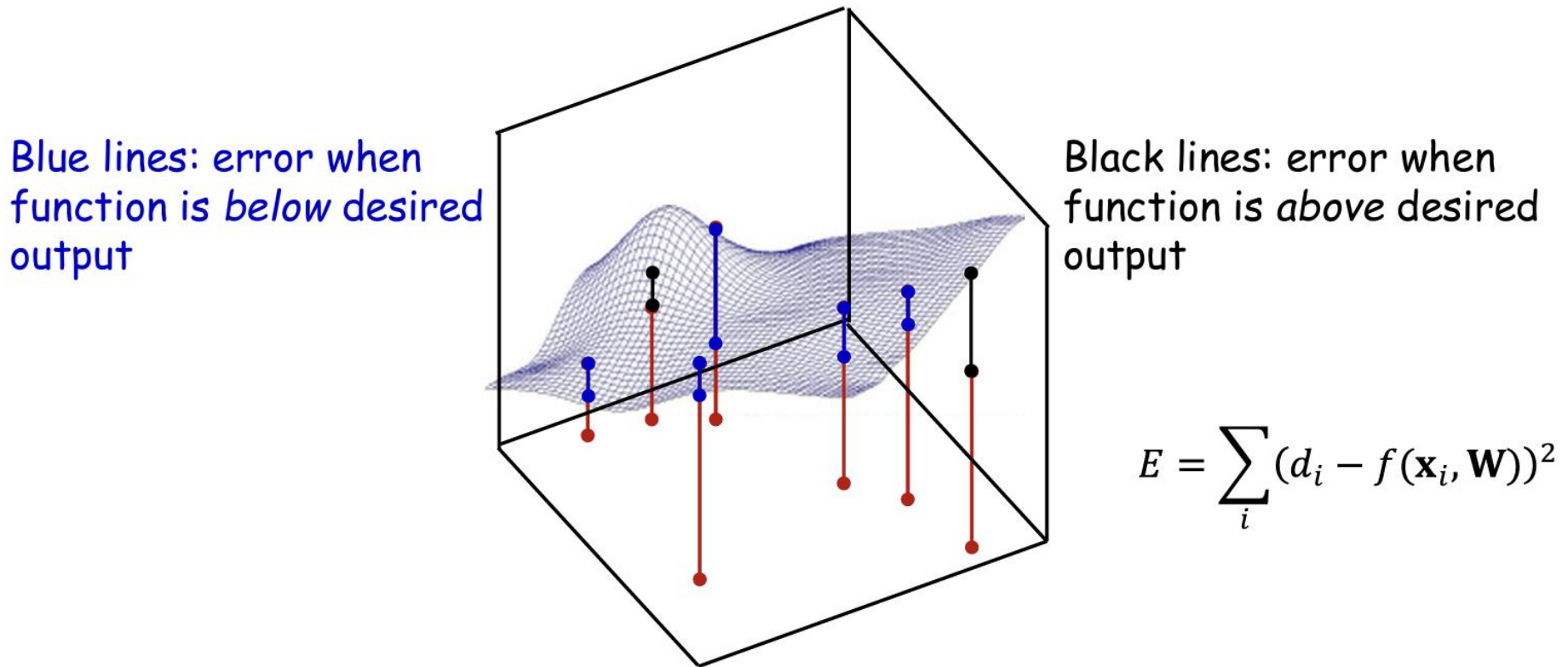


Learning a ML algorithm



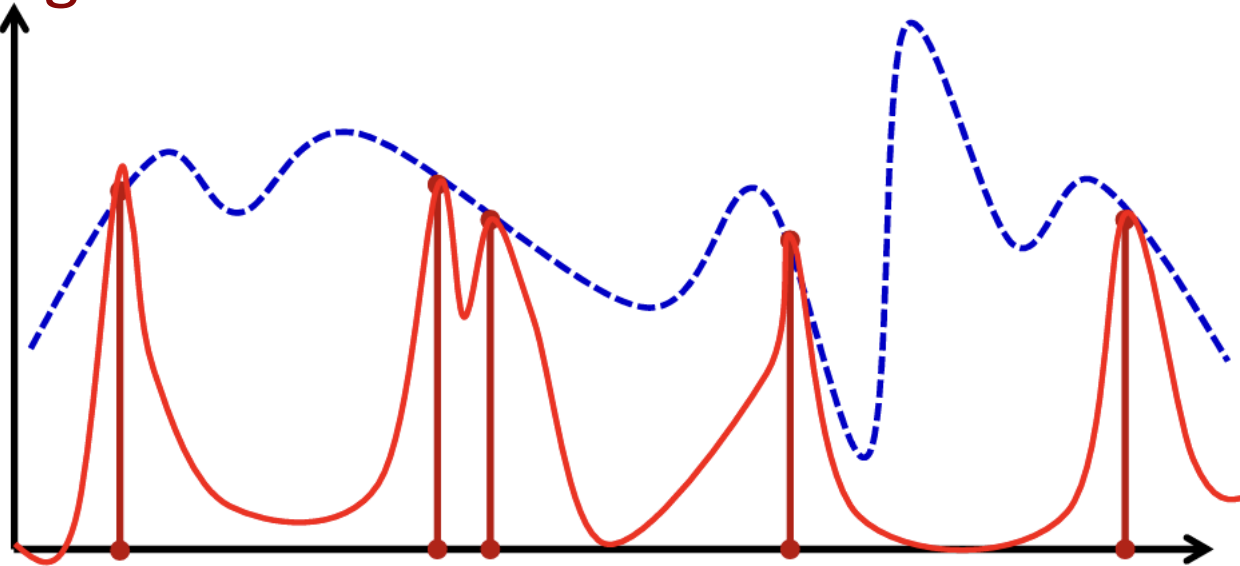
- We attempt to learn an entire function from just a few snapshots of it

General Approach to Supervised Learning



- Define a **divergence** between the actual output for any parameter value and the desired output
- Typically L2 divergence or KL divergence

Overfitting



- Problem: ML algorithms may just learn the values of the inputs if the number of parameters in a model is **a lot more than** the true function
- Learn the red curve instead of the dotted blue one
 - Given only the red vertical bars as inputs
- **The loss on a new unseen datapoint will be large!**

Example: polynomial regression

- In a 1D polynomial regression

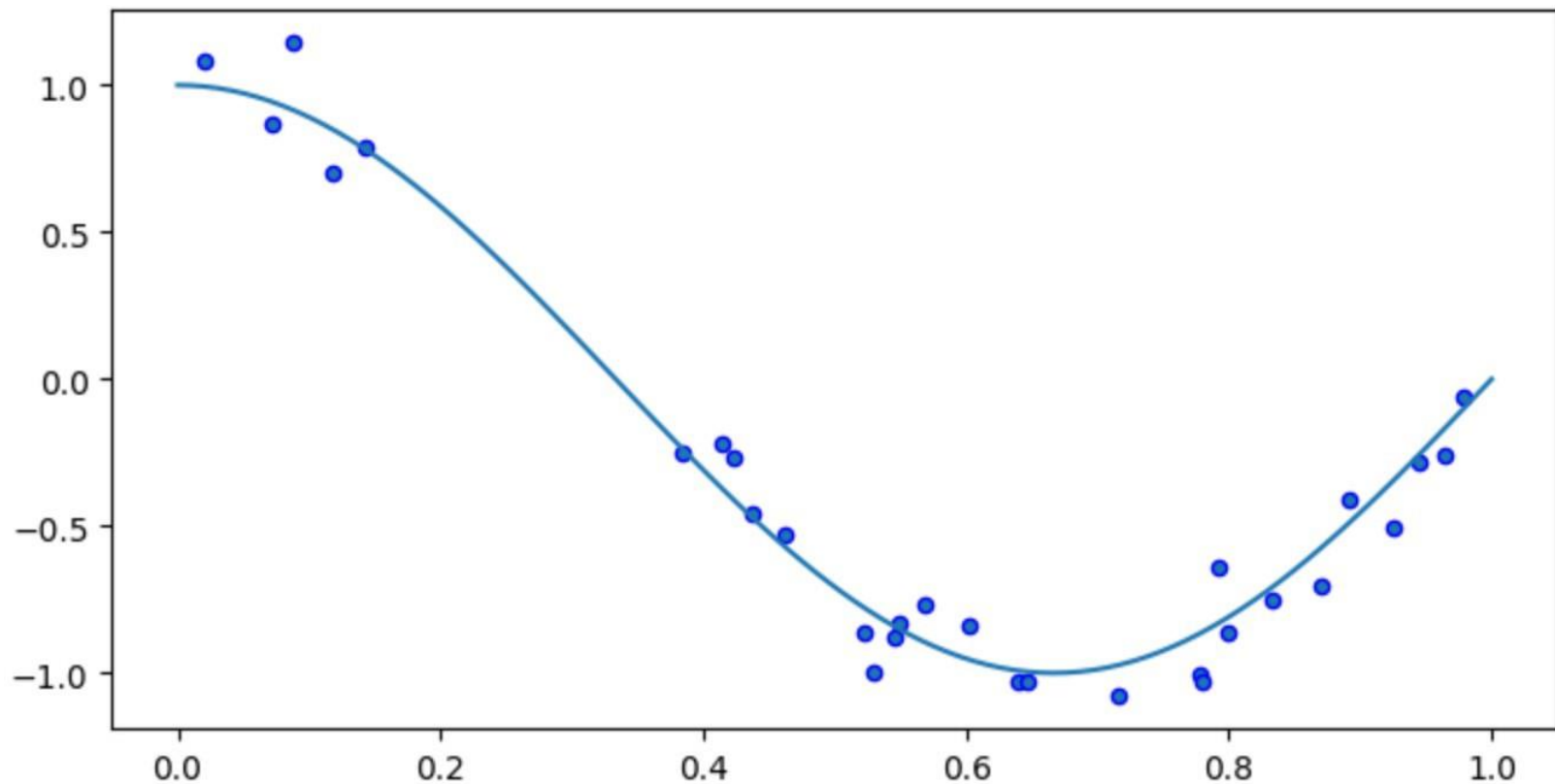
- Given data $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$

- Fit model:

$$f_{\theta}(x) = \theta_1 + \theta_2 x + \theta_3 x^2 + \dots + \theta_{p+1} x^p$$

- Polynomials of degree p
- Compared with linear model, polynomial fits the data better

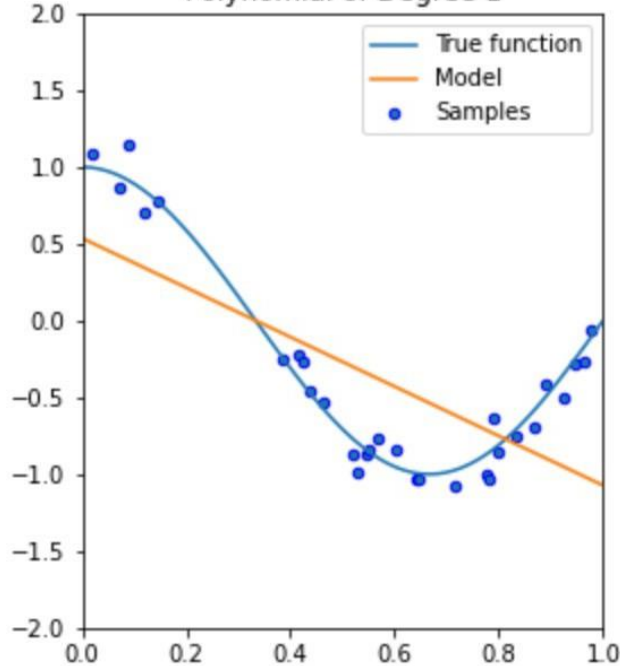
- Generate 30 samples from an unknown true generating function `true_fn`
- $X, \epsilon \sim \text{Unif}(0,1)$
- Plot true function, and data points generated



- Fits polynomials of degree 1, 2, and 3
- Data is fixed

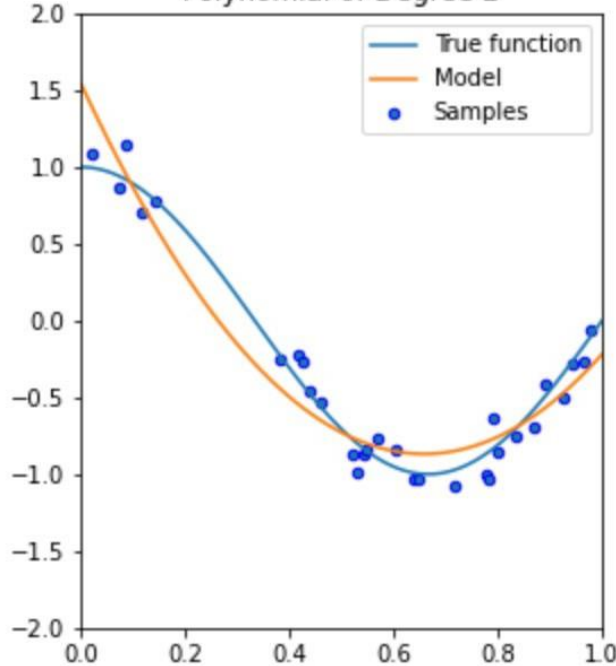
`[-7.31956683 5.55955392]`

Polynomial of Degree 1



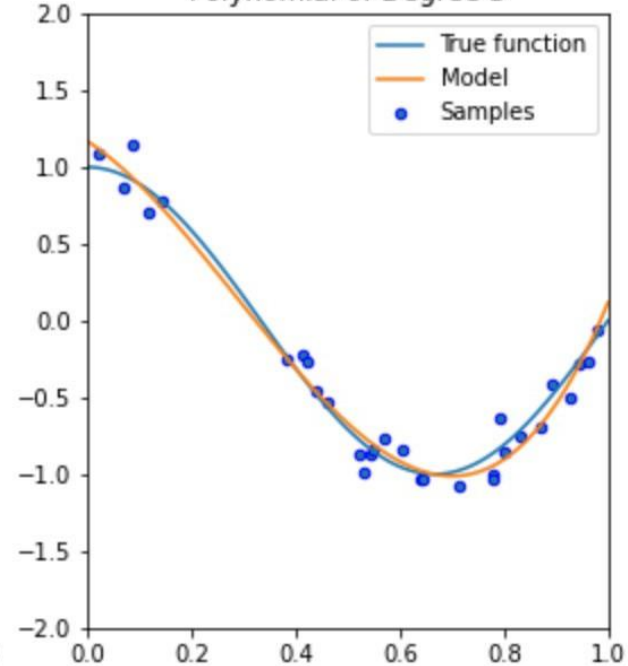
`[-1.60931179]`

Polynomial of Degree 2

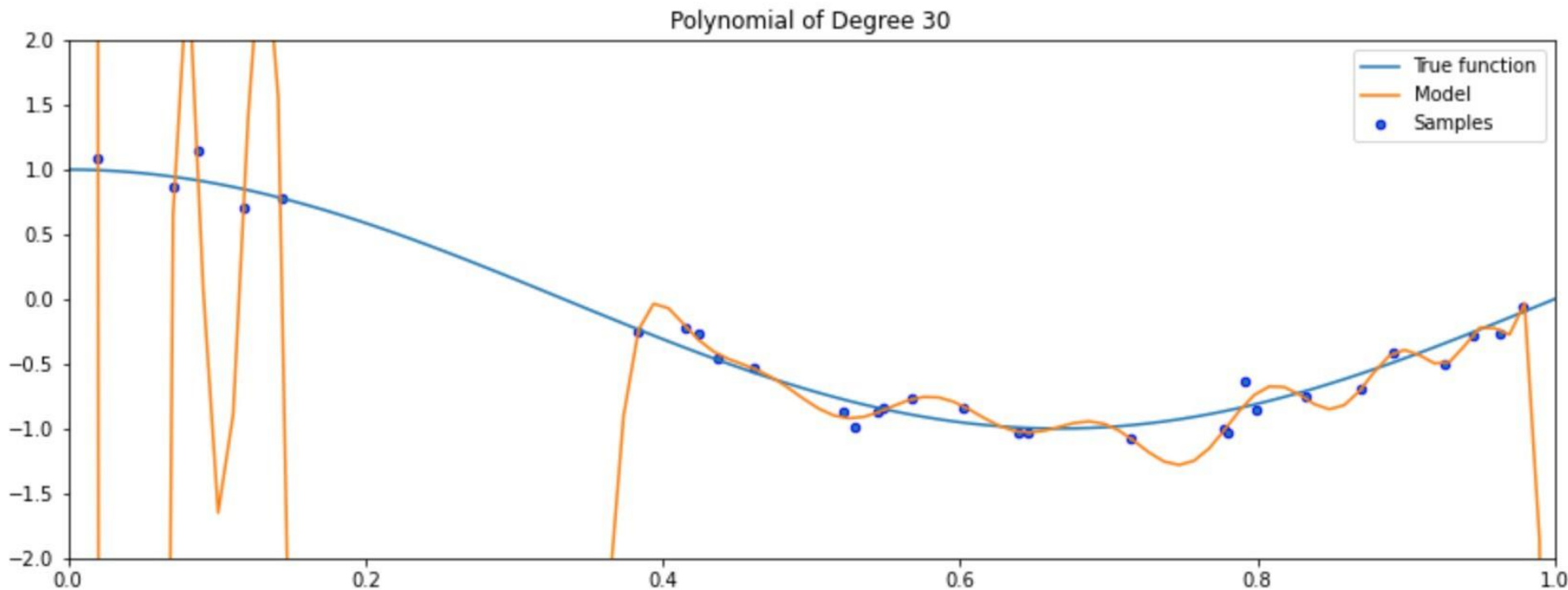


`[-2.19617614 -7.05669992 8.202858 ]`

Polynomial of Degree 3



Polynomial Degree =30



```
[ -2.05371833e+05  8.39101550e+06 -1.82675334e+08  2.43454110e+09
 -2.13281315e+10  1.27772971e+11 -5.33268720e+11  1.54670466e+12
 -3.00976103e+12  3.48898042e+12 -1.25714807e+12 -2.23130433e+12
  2.26319837e+12  1.49429817e+12 -2.24798963e+12 -1.58155550e+12
  1.85771643e+12  2.05724745e+12 -1.09322793e+12 -2.46532383e+12
  1.17505644e+10  2.49513783e+12  1.07553470e+12 -2.17845426e+12
 -1.83745331e+12  2.05110612e+12  2.01948345e+12 -3.41371839e+12
  1.66729405e+12 -2.87952109e+11]
```

Are the following statements on generalization correct?



The generalization error measures the performance of the model on unseen data, and it can be approximated by holding out a portion of the data during the training

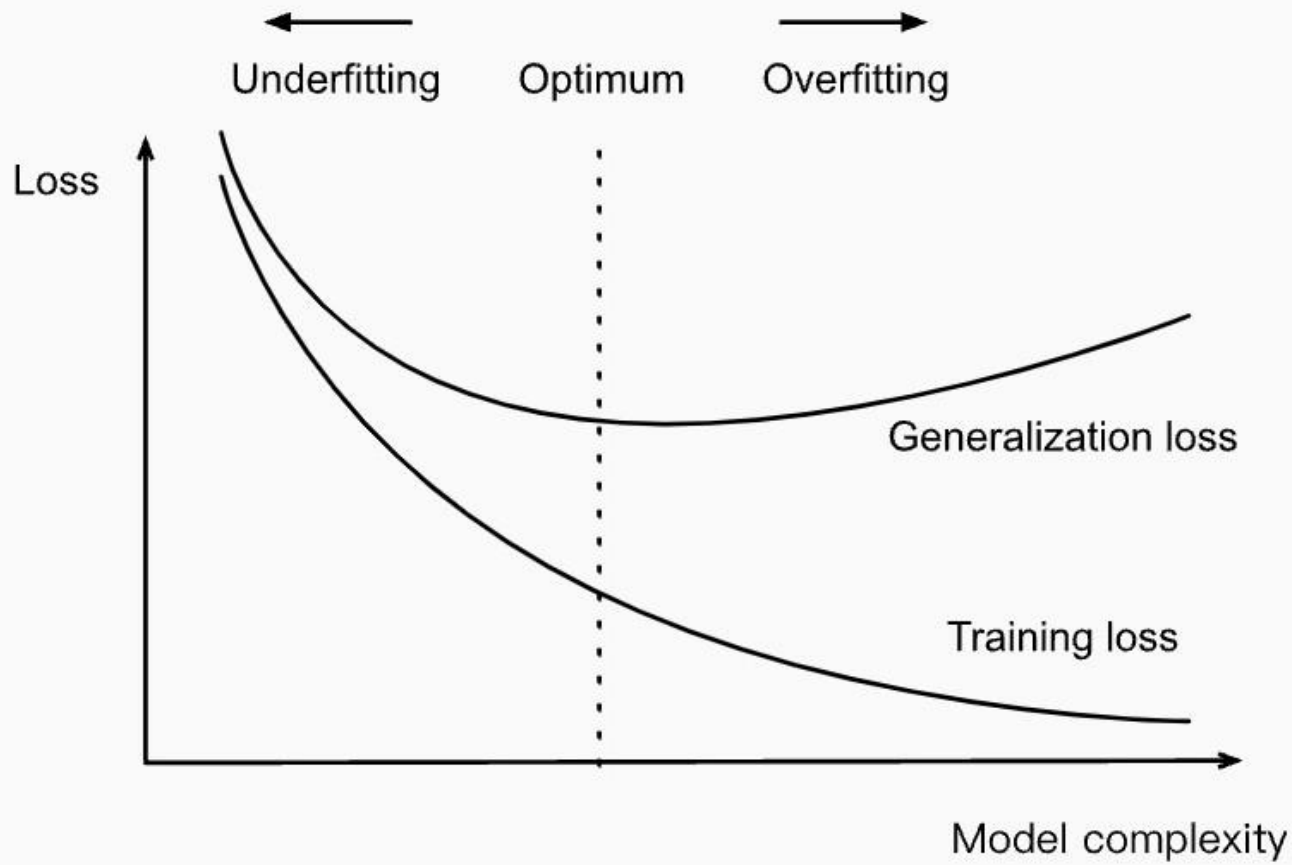
0

By splitting the dataset into training and testing sets, we can determine whether the current model overfits the testing dataset by comparing its performance on both the training and testing datasets.

0

Training and generalization error depends on model complexity.

0



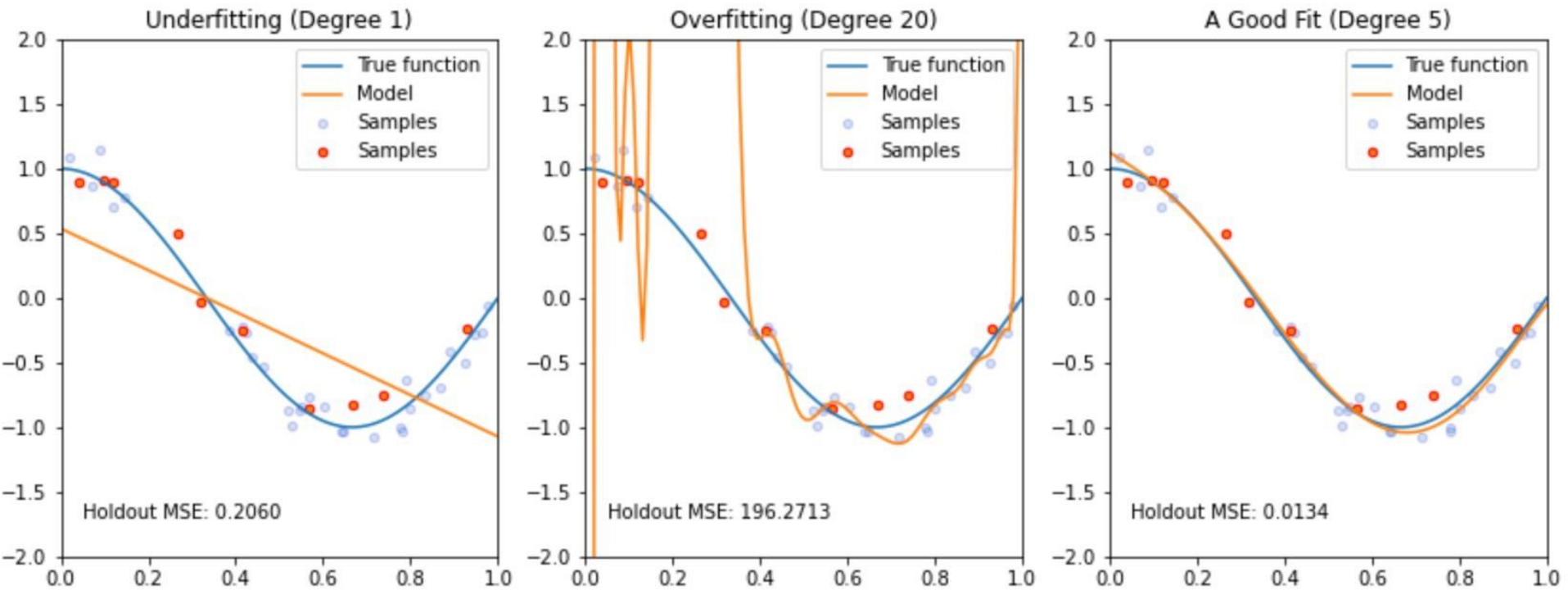
Underfitting: generalization error could be reduced further by $\downarrow$ the training error

- Happens when training and testing errors are close
- Model is overly simple

Overfitting: $\downarrow$ training error results in $\uparrow$ generalization error

- Training error $\ll$ testing errors
- Too many model parameters

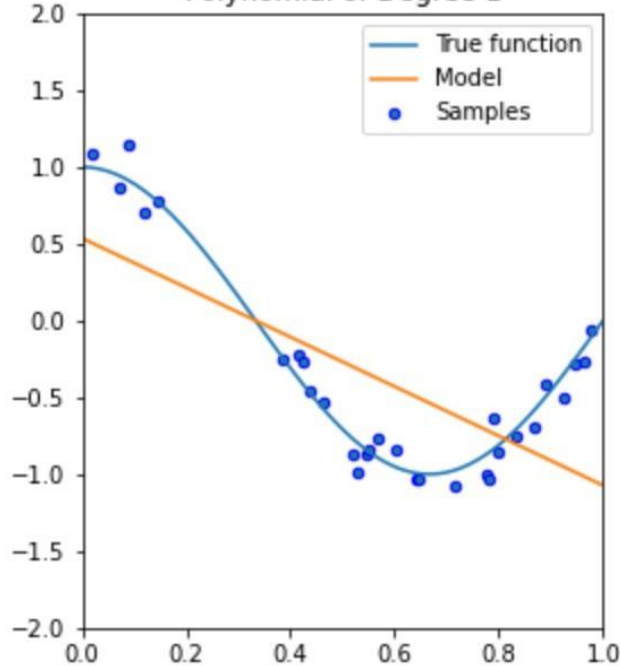
Overfitting vs Underfitting: Polynomial Regression



- Fits polynomials of degree 1, 2, and 3
- Data is fixed

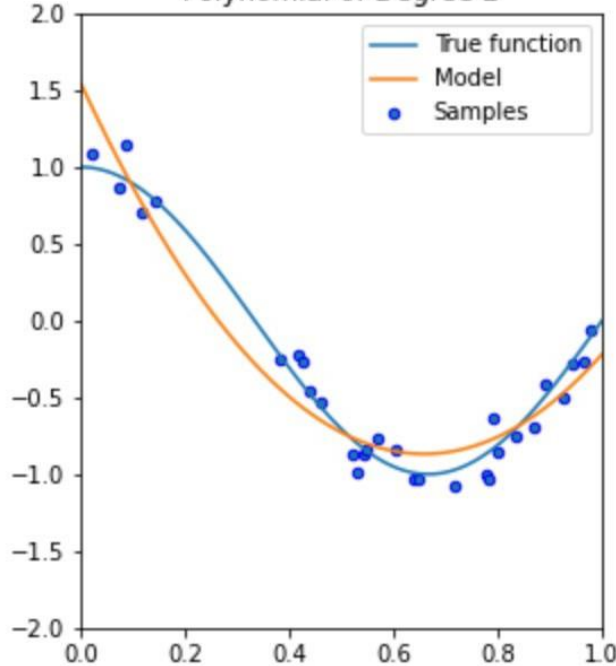
`[-7.31956683 5.55955392]`

Polynomial of Degree 1



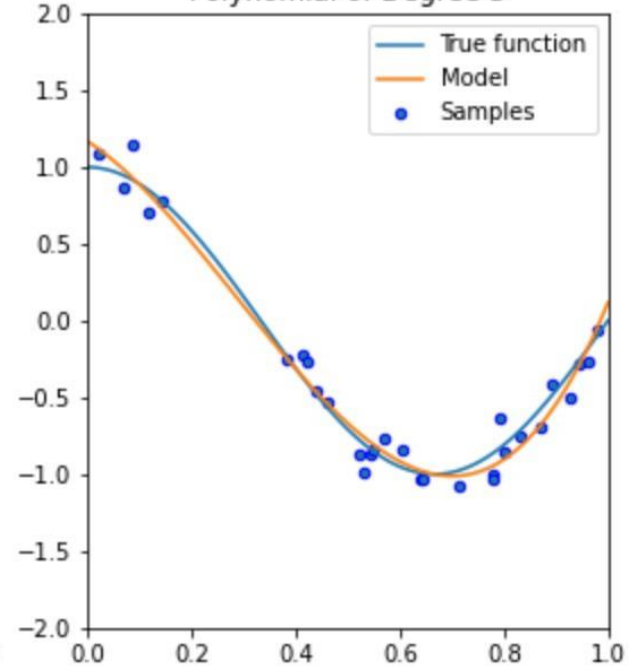
`[-1.60931179]`

Polynomial of Degree 2

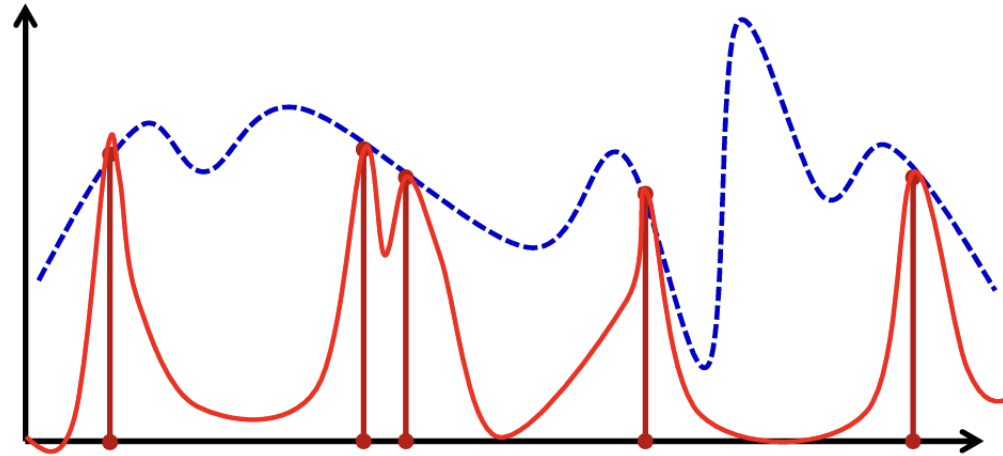


`[-2.19617614 -7.05669992 8.202858 ]`

Polynomial of Degree 3

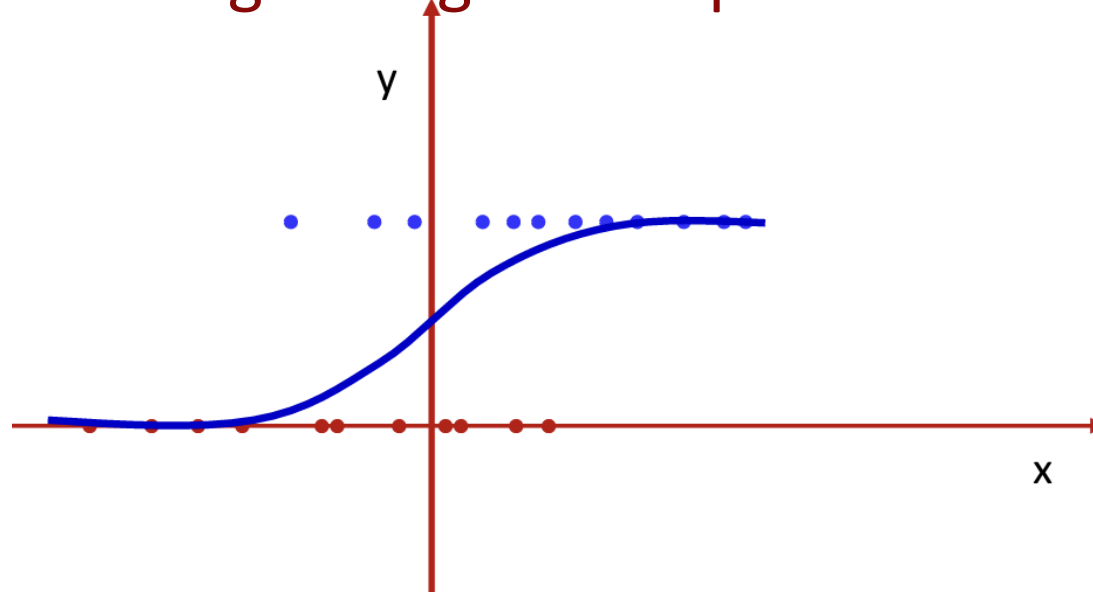


Overfitting



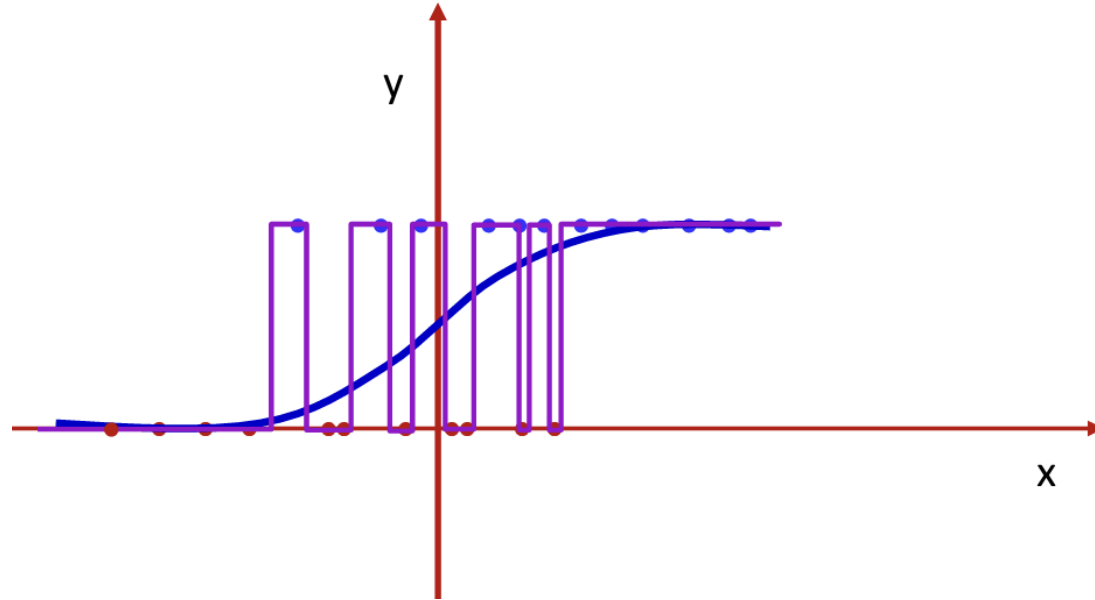
- Problem: ML algorithms may just learn the values of the inputs if the number of parameters in a model is **a lot more than** the true function
- Need additional “**smoothing**” constraints that will “fill in” the missing regions acceptably
- Generalization

Smoothness through Weight Manipulation



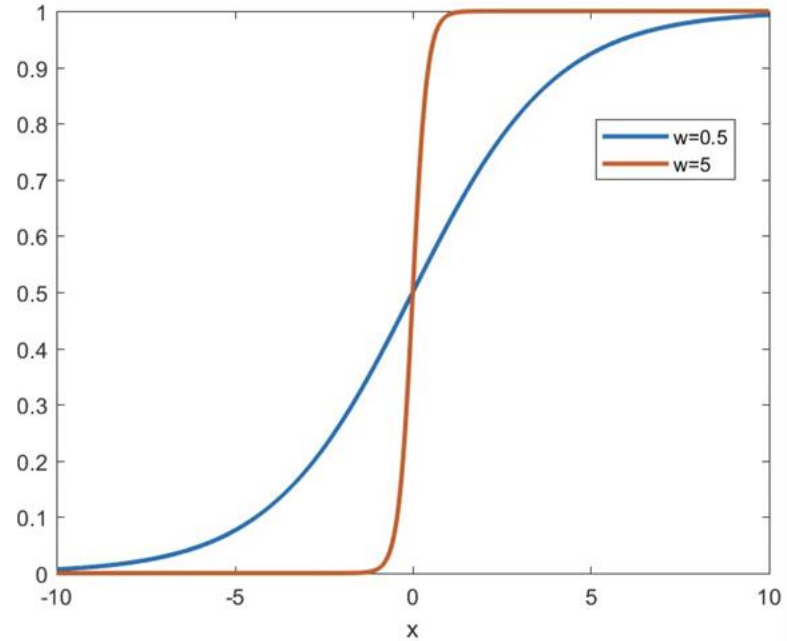
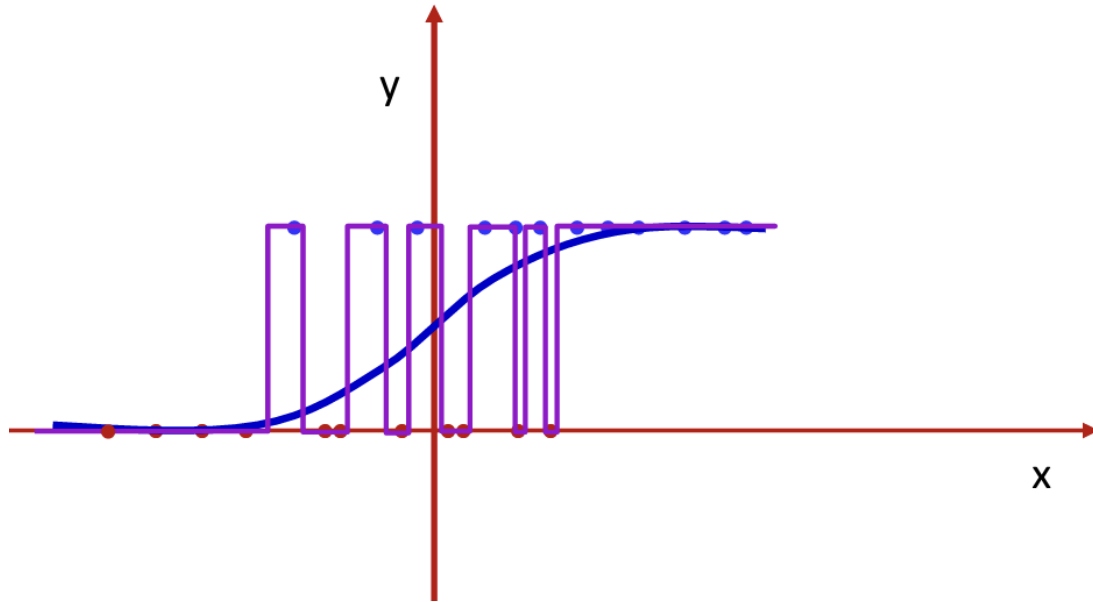
- Illustrative example: Simple binary classifier
 - E.g., binary classification: logistic regression
 - The “desired” output is generally smooth
 - Capture statistical or average trends

Smoothness through Weight Manipulation



- Illustrative example: Simple binary classifier
 - The “desired” output is generally smooth
 - Capture statistical or average trends
 - An unconstrained model will model individual instances instead
 - E.g., in binary classification: running a high-degree polynomial regression first and then passing it through a sigmoid function

Smoothness through Weight Manipulation



- Steep changes that enable overfitted responses are facilitated by over-parametrized models with large parameters
- Constraining the parameters to be low will force smoother output response

Are the following statements on generalization correct?

The solution to overfitting is to use 1) more features than existing ones, 2) more expensive model family (poly vs linear regression), and 3) better optimization...

0

The solution to underfitting is to use 1) simpler model class, 2) collect more data, and 3) modify the training process to penalize overly complex models

0

When being thought of as global evaluation metrics, overfitting and underfitting cannot happen at the same time.

0

Objective for Supervised Learning

Training loss:

$$\ell(f_\theta) = \underbrace{\frac{1}{n} \sum_{i=1}^n \text{Div} \left(y^{(i)}, f_\theta(x^{(i)}) \right)}_{\text{Learning Objective}}$$

Conventional training: minimizing the loss:

$$\hat{\theta} = \arg \min_{\theta} \ell(f_\theta)$$

Smoothness through weight constraints:

- Minimize the loss while also minimizing the weights

Smoothness Through Weight Constraints: L2 Regularization

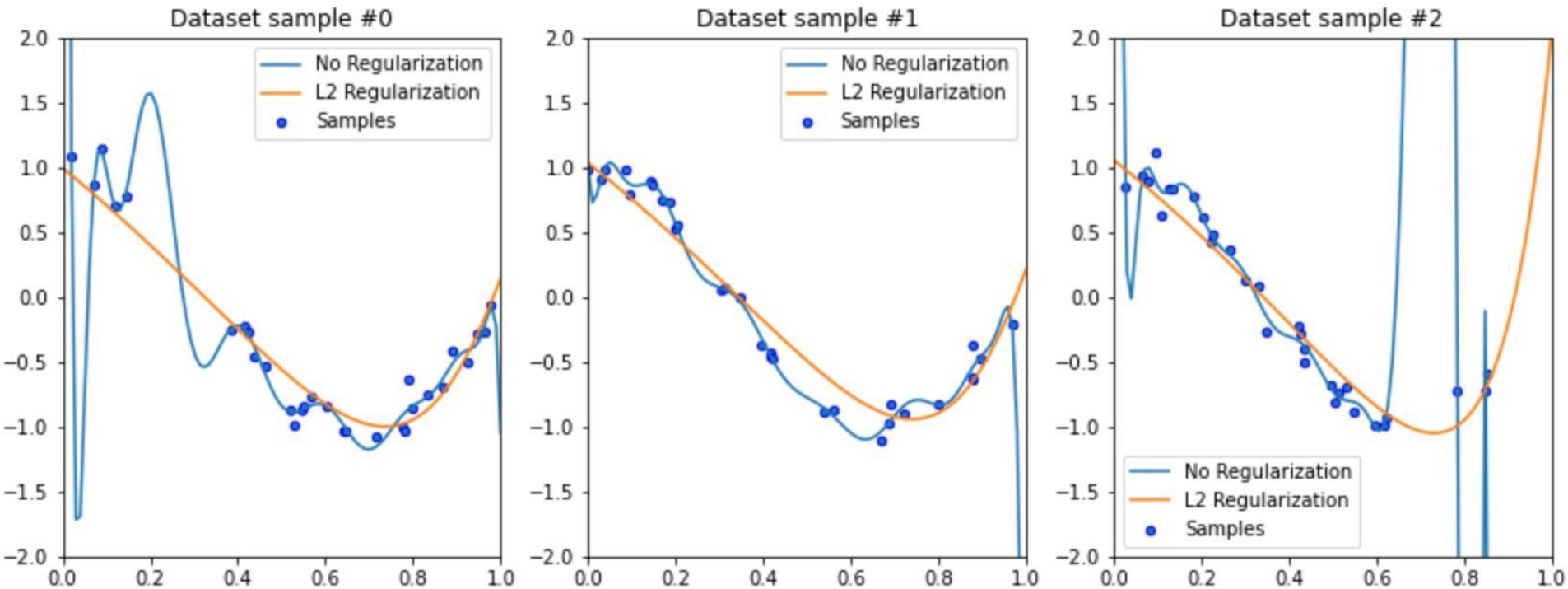
- Regularized loss function:

$$\ell(f_\theta) = \underbrace{\frac{1}{n} \sum_{i=1}^n \text{Div} \left(y^{(i)}, f_\theta(x^{(i)}) \right)}_{\text{Learning Objective}} + \underbrace{\frac{1}{2} \lambda \|\theta\|_2^2}_{\text{L2 penalty}}$$

- $\|\theta\|_2^2 = \sum_{j=1}^d \theta_j^2$
- λ : tunable regularization parameter
- Increasing λ assigns greater importance to shrinking the parameters
- Making greater error on training data, to obtain more acceptable model parameters

L2 Regularization Example

Uniformly reduces weights to prevent high-degree polynomials from learning irregular functions



Non-regularized weights of the polynomial model need to be large to fit every point:
[-3.02370887e+03 1.16528860e+05 -2.44724185e+06 3.20288837e+07]

By regularizing the weights to be small, we force the curve to be more regular:
[-2.70114811 -1.20575056 -0.09210716 0.44301292]

How to Choose λ ? Hyperparameter Search

- λ : hyperparameter/tunable parameter
 - A high-level parameter that controls the value of other parameters
- How to choose λ ?
 - Select the λ that performs the best on the development set
 - Select λ by cross-validation (more on this later)

L2 Regularization in Linear Regression Can be Solved in Closed-Form

L2-regularized least square objective in linear regression:

$$\ell(\theta) = \frac{1}{2} (X\theta - y)^T (X\theta - y) + \frac{1}{2} \lambda \|\theta\|_2^2$$

- Take gradient and set it to zero
- Optimal $\theta^* = (X^T X + \lambda I)^{-1} X^T y$
 - $X^T X + \lambda I$: always invertible
- Also known as the ridge regression

L2 Regularized Update in Gradient Descent

- L2-regularized general objective:

$$\ell(\theta) = \frac{1}{n} \sum_{i=1}^n \text{Div} \left(y^{(i)}, f_{\theta}(x^{(i)}) \right) + \frac{1}{2} \lambda \|\theta\|_2^2$$

- Gradient update:

$$\nabla_{\theta} \ell(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla_{\theta} \text{Div} \left(y^{(i)}, f_{\theta}(x^{(i)}) \right) + \lambda \theta^T$$

L1 Regularization (Lasso)

- L2 penalty term: $\frac{1}{2} \lambda \|\theta\|_2^2$
- L1 regularization (L1 norm of θ):

$$\ell(\theta) = \frac{1}{n} \sum_{i=1}^n \text{Div}(y^{(i)}, f_{\theta}(x^{(i)})) + \lambda \cdot \|\theta\|_1$$

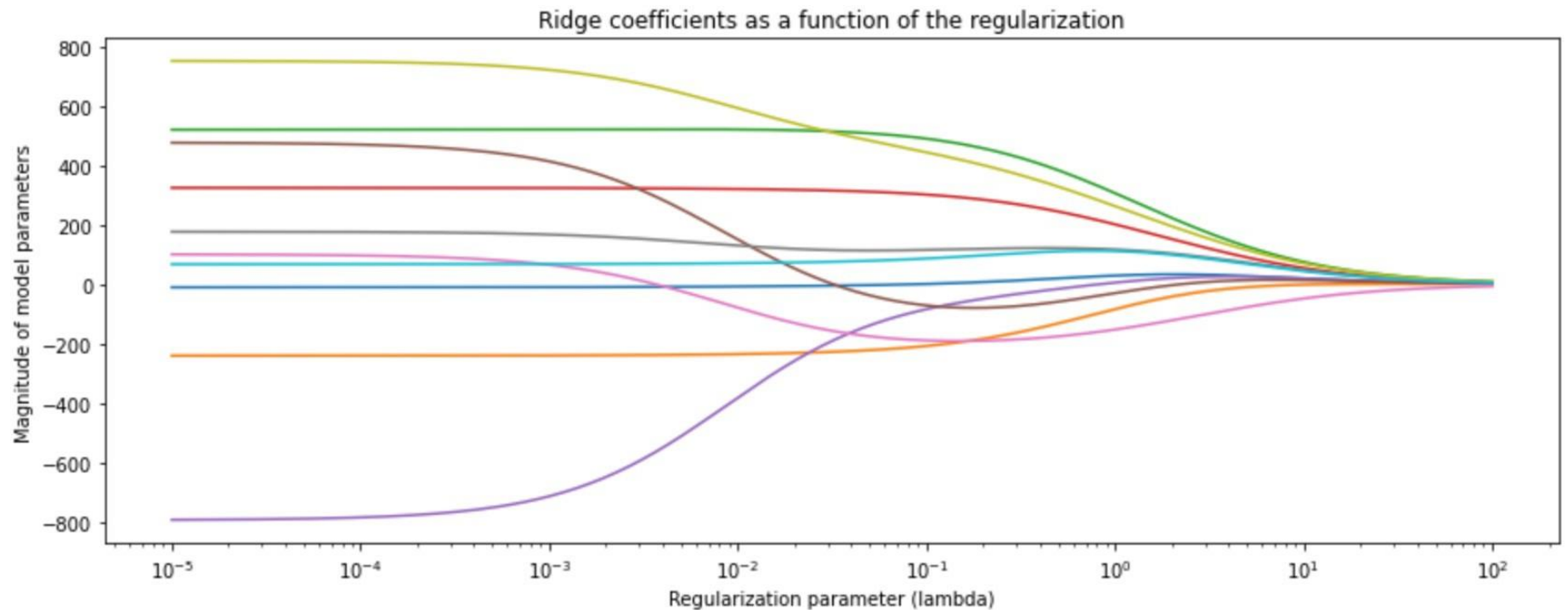
- $\|\theta\|_1 = \sum_{j=1}^d |\theta_j|$
- Forces weights to decay to zero rather than being small
- Optimizer: subgradient descent, coordinate descent, least angle regression, and etc.

Sparsity

- Definition: a vector is said to be sparse if a large fraction of its entries is zero.
- Lasso regression introduces sparsity on θ

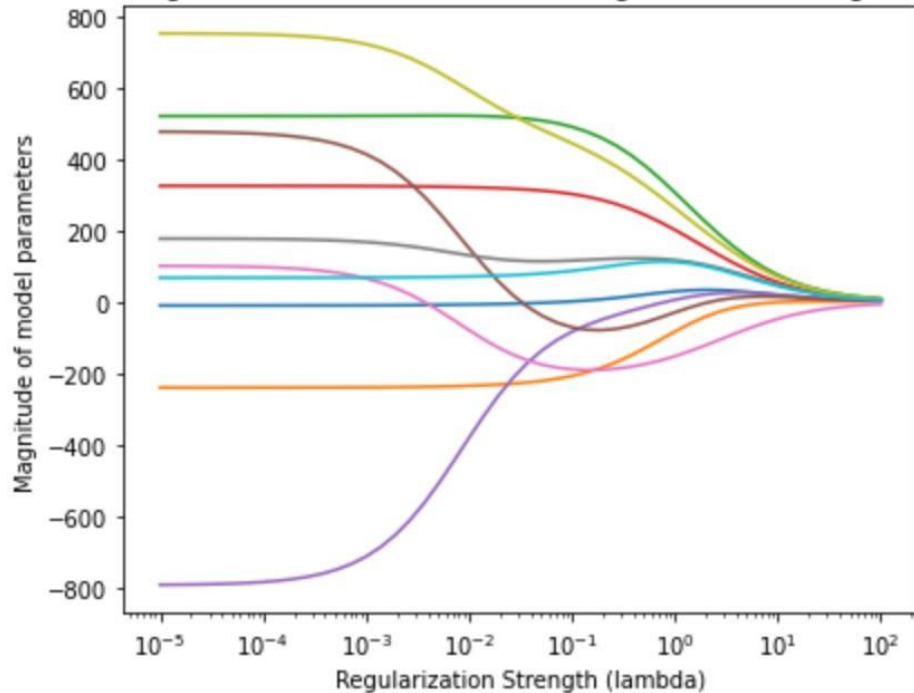
UCI diabetes dataset

- Magnitude of weights (represented by lines) as a function of the regularization parameter

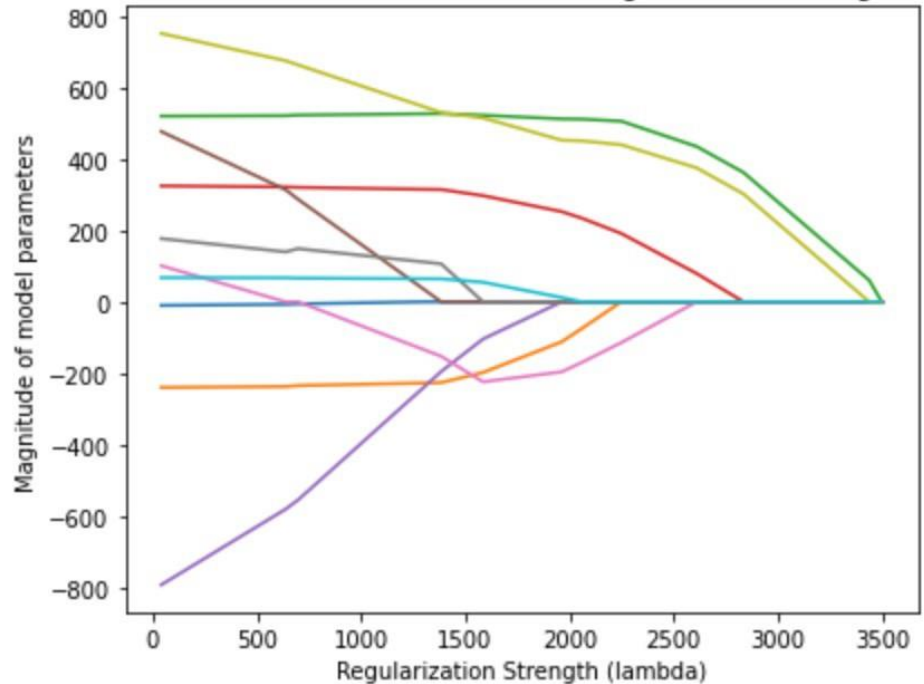


- Lasso parameters become progressively smaller
- Stay at zero when the parameters reach 0

Ridge coefficients as a function of regularization strength λ



LASSO coefficients as a function of regularization strength λ



Regularizing via Constraints: example

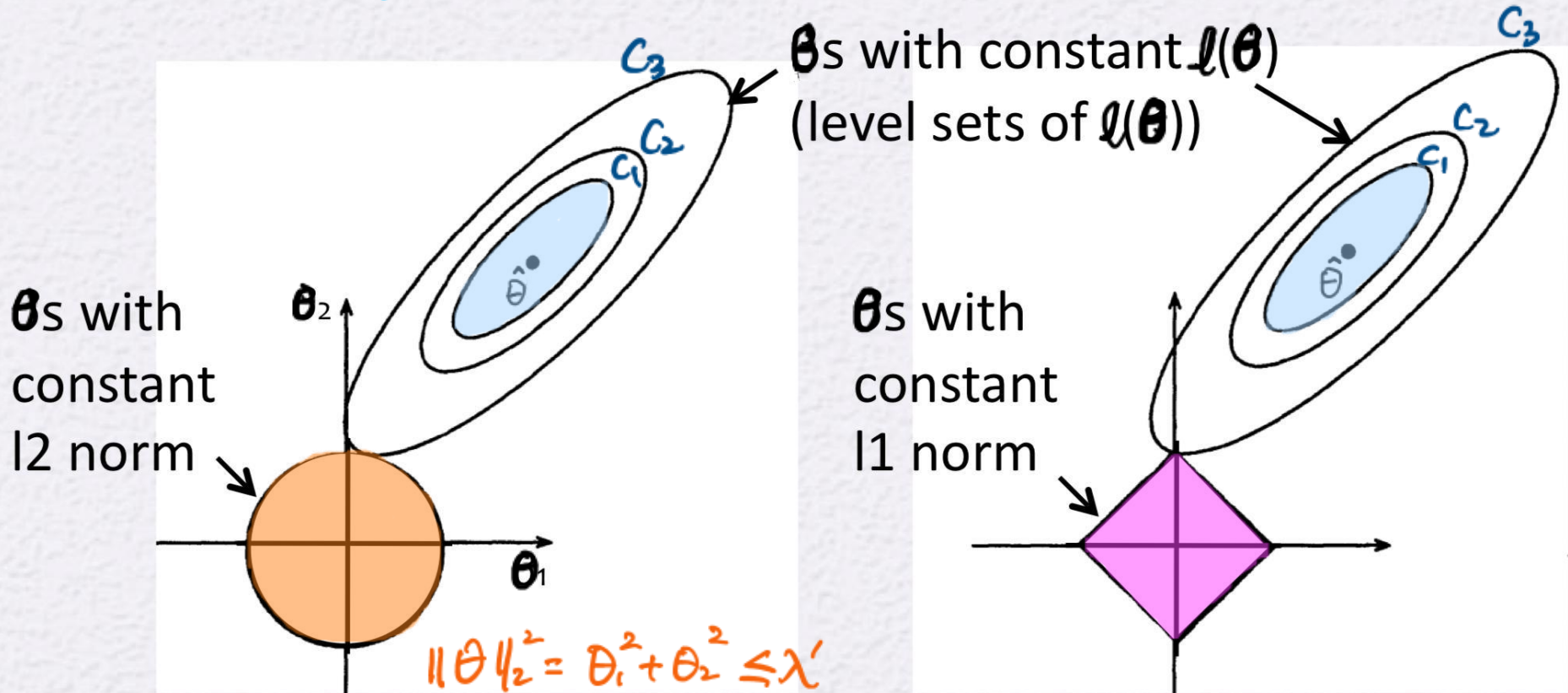
We can view regularization as a constrained optimization problem:

$$\min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n \text{Div}(y^{(i)}, f_{\theta}(x^{(i)}))$$

such that $\|\theta\| \leq \lambda'$

- $\|\cdot\|$: either L1 or L2 norm

$$\frac{1}{n} \sum_{i=1}^n D_{iv}(y^{(i)}, f_{\theta}(x^{(i)})) = C_{\tilde{v}} \text{ for some constant } C_{\tilde{v}}$$



$$\|\theta\|_1 = |\theta_1| + |\theta_2| \leq \lambda'$$

optimization objective: find the smallest $C_{\tilde{v}}$ such that $\bigcirc$ intersects $\bigcirc$ or $\blacklozenge$